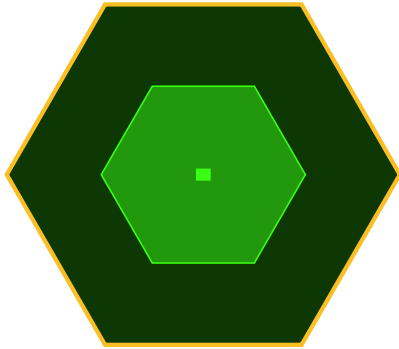


Athera Secure (SentinelAPI): Zero-Trust API Vulnerability Scanning via Cross-Principal Differential Testing

Team Athera Secure
AMIHACKS 2026 — Track C (Industry / Deep-Tech)
Jaipur, Rajasthan, India

Live App: <https://athera-secure-production.up.railway.app>
GitHub: <https://github.com/kanishksharma76460-design/sentinelapi>



Abstract—Broken object-level authorization (BOLA/IDOR) and excessive data exposure are the two most damaging classes of API vulnerability, yet most teams review API security manually, infrequently, and only after deployment. We present Athera Secure (SentinelAPI), a production-deployed, zero-trust API vulnerability scanner that detects fourteen vulnerability classes from an OpenAPI specification and live traffic: BOLA/IDOR, excessive data exposure, broken authentication, broken function-level authorization (BFLA), mass assignment, security misconfiguration, missing rate limiting, exposed debug endpoints, JWT misconfiguration, SQL and NoSQL injection, CORS misconfiguration, server-side request forgery (SSRF), GraphQL introspection, and undocumented endpoints. The core of the approach is *differential testing across security contexts* for authorization, a *schema-contract oracle* for data over-exposure, and *crafted active probes* (alg:none JWT forging, SQL/NoSQL payloads, CORS origin reflection, SSRF loopback/metadata, GraphQL introspection) for the remaining classes. The engine is implemented in Go (a single static binary) and wrapped in a hardened FastAPI product (SSRF guard, rate limiting, optional session login, bounded job pool, cancel/delete, HTML/JSON/PDF/SARIF/CSV reports, WebSocket live logs, regression diff, webhooks) behind a geometric consent gateway rendered as a robotic hexagonal-aperture iris (neon-green/gold visual identity, boot telemetry, and a heads-up-display attack console). Every scan is graded A–F and mapped to OWASP API Security Top 10 (2023) with remediation guidance, and the attack surface is rendered as an interactive graph. The system is verified against two deliberately vulnerable sandbox APIs with precision = recall = F1 = 1.0 (42/42 findings, zero false positives, zero false negatives), and is live at <https://athera-secure-production.up.railway.app>.

Index Terms—API security, broken object-level authorization, differential testing, OpenAPI, vulnerability scanner, OWASP API Top 10, JWT, SQL injection, SSRF

I. INTRODUCTION

Modern applications are assembled from dozens to hundreds of internal and third-party APIs. Breaches that exploit broken object-level authorization (“API1:2023”) and excessive data exposure are a leading cause of production data leakage [1]. The defect is subtle: a request can be syntactically valid and authenticated yet still return another user’s data, because the flaw lives in *authorization logic*, not request formatting. Signature-based web scanners miss this class entirely; enterprise API-security platforms are inaccessible to smaller teams; and manual penetration testing does not scale with deployment velocity.

Athera Secure closes this gap with a lightweight, deterministic scanner whose correctness is verifiable by construction. It encodes the API’s *security contract* as a set of oracles and tests the contract against observed behavior, rendering explainable, severity-ranked findings with copy-paste reproduction, an OWASP mapping, and a remediation recommendation. The full stack—Go engine, FastAPI backend, and web dashboard—is deployed on Railway and scans a live, publicly reachable sandbox in seconds.

II. THREAT MODEL

We assume an attacker who is an *authenticated but unauthorized* user: they possess a valid token but should not be able to read or modify other principals’ objects. This matches the most common real-world API breach profile. We further assume the target exposes an OpenAPI/Swagger specification (the norm for modern APIs) and that the scanner operates with the owner’s consent. The deployed product therefore gates every scan behind an explicit geometric consent gateway and refuses targets that resolve to private, loopback, link-local, or reserved addresses (SSRF guard), preventing the scanner itself from being weaponized against internal networks.

III. RELATED WORK AND STATE OF THE ART

Differential testing [2] runs the same input through two or more contexts and treats divergence as a defect. SentinelAPI applies this to security: the same endpoint is executed under

two principals, and any difference in data ownership is a defect.

RESTler [3] generates stateful sequences of API calls from a Swagger/OpenAPI spec to find reliability bugs. *EvoMaster* [4] applies evolutionary search to REST API test generation, and *Morest* [5] adds model-based state fuzzing. Commercial scanners such as OWASP ZAP and 42Crunch detect syntactic and configuration issues but largely miss the authorization-logic class that dominates real breaches. These tools target reliability, coverage, or configuration; our goal is narrower and our oracle is stronger—we test a specific, well-defined authorization property rather than searching for crashes. Table I contrasts the approach.

TABLE I
STATE-OF-THE-ART COMPARISON

Tool	AuthZ	Schema	Active probes	Explainable	Grade
RESTler [3]	—	partial	—	—	—
EvoMaster [4]	—	partial	—	—	—
OWASP ZAP	partial	—	partial	partial	—
42Crunch	partial	full	—	partial	—
Athera Secure	full	full	full	full	A-F

IV. SYSTEM ARCHITECTURE

The pipeline is: *parse OpenAPI* $\rightarrow$ *register principals* $\rightarrow$ *generate tests* $\rightarrow$ *execute across security contexts* $\rightarrow$ *oracle evaluation* $\rightarrow$ *severity-ranked, graded, OWASP-mapped report*. The engine is isolated behind a command-line boundary (`--base` $\rightarrow$ `findings.json` + `report.html`), so orchestration is language-agnostic. The product wraps the engine in a FastAPI service exposing a consent-gated, SSRF-guarded, rate-limited, and API-key-authenticated scan API with a web dashboard.

A. Repository layout

Listing 1. Folder architecture (code only)

```
sentinelapi/
|-- backend/
|   |-- main.py           # FastAPI product backend
|   |                   # scan API, SSRF guard, auth,
|   |                   # rate-limit, job pool,
|   |   reports
|   |-- requirements.txt
|   |-- static/
|   |   |-- index.html    # dashboard UI (globe, grade,
|   |                   # attack map, findings)
|-- engine/               # Go scanner engine (static
|   binary)
|   |-- main.go           # CLI entry + --checks selector
|   |-- scanner.go        # 8 core checks (authz /
|   |                   exposure)
|   |-- checks_extra.go   # 6 active probes (JWT, SQLi,
|   |                   # CORS, SSRF, GraphQL, spec)
|   |-- oracle.go         # identifier inference + schema
|   |   paths
|   |-- spec.go           # OpenAPI fetch / parse
|   |-- report.go         # findings.json + report.html
|   |-- go.mod
|-- frontend/             # React + Three.js dashboard
|   |-- src/              # gate, boot, HUD, findings,
|   |   guide
|-- scanner/
|   |-- scanner.py         # Python reference mirror
|-- vuln_api/
```

```
|   |-- main.py           # sandbox 1: "Bank" (32 seeded
|   |                   flaws)
|-- vuln_api2/
|   |-- main.py           # sandbox 2: "Library" (10
|   |                   seeded flaws)
|-- tests/
|   |-- accuracy.py       # precision/recall/F1 benchmark
|   |-- e2e_backend.py    # end-to-end product test
|-- Dockerfile            # product image (Go + Python)
|-- Dockerfile.sandbox   # sandbox image
|-- railway.toml          # deploy + healthcheck config
|-- run.sh
```

B. Deployment

The product and the deliberately vulnerable sandbox are deployed as two Railway services and are publicly reachable:

- Product (scanner + dashboard): <https://athera-secure-production.up.railway.app>
- Sandbox (vulnerable target): <https://athera-secure-sandbox-production.up.railway.app>
- Source code (private repository): <https://github.com/kanishksharma76460-design/sentinelapi>

V. DETECTION METHOD

Each vulnerability class is encoded as a deterministic oracle. The scanner registers two synthetic principals *A* and *B*; registration returns each principal’s identifying fields, inferred generically from the response rather than hardcoded, so the scanner is target-agnostic.

A. BOLA / IDOR (cross-principal differential oracle)

For every endpoint *e* with a path parameter *p*, the scanner substitutes each of *B*’s identifiers *v* into *p* while presenting *A*’s token. Writing $R_v^A = R(A, e[p:=v])$ for the response under *A*’s session with *v* substituted, a leak is detected when:

$$\text{leak}(e, v) = 200(R_v^A) \wedge \exists m \in M_B : m \in R_v^A \quad (1)$$

where M_B is the set of *B*’s identifying values. A satisfied predicate flags a CRITICAL finding. The oracle inspects *content*, not status code alone—a 404 is safe behavior, whereas a 200 containing foreign data is a leak.

B. Excessive data exposure (schema-contract oracle)

The specification declares, for each endpoint, a response schema—the fields the server *promises* to return. The scanner computes the set difference between the paths actually returned and the paths declared, recursing into nested objects:

$$\text{extra}(e) = \text{paths}(R(A, e[p:=id_A])) \setminus \text{schemaPaths}(e) \quad (2)$$

Any undeclared path whose leaf is sensitive (password, ssn, card, token, `library_card`, ...) is flagged HIGH; others MEDIUM.

C. Broken authentication oracle

When the specification declares an API-key security requirement for an endpoint, the scanner probes the endpoint with *no* Authorization header. If the endpoint returns data (HTTP 200), the declared security is not enforced.

D. BFLA and mass assignment

For endpoints under an administrative path prefix, the scanner probes with a regular (non-admin) token; a 200 response indicates a missing role check. For write endpoints, it injects privileged fields (`role`, `is_admin`) and flags any endpoint that accepts and reflects them.

E. Security misconfiguration, rate limiting, and debug endpoints

Three API-wide checks complement the per-endpoint oracles. The scanner inspects response headers for server-version disclosure (`X-Powered-By`, `Server`) and for missing hardening headers (`X-Content-Type-Options`, `X-Frame-Options`, `HSTS`). It floods a cheap endpoint with a burst of requests and flags the absence of any 429/Retry-After response (OWASP API4: unrestricted resource consumption). Finally, it probes a curated list of debug/inventory paths (`/debug`, `/metrics`, `/.env`, `/actuator`, `/admin/debug`, ...) and flags any that leak internal information (OWASP API9: improper inventory management).

F. JWT analysis

When the target issues JSON Web Tokens (RFC 7519) [6], the scanner harvests a token from login/registration responses and tests three properties: (i) **signature verification** by forging an `alg:none` header and presenting it to a protected endpoint; (ii) **key strength** by re-signing the token with a dictionary of weak HMAC secrets; and (iii) **expiry** by checking for a missing `exp` claim. Acceptance of any forged token is flagged CRITICAL/HIGH.

G. SQL and NoSQL injection

GET endpoints receive error-based SQL probes (`'`, `"`, `1' OR '1'='1`) whose responses are matched against a dictionary of database error signatures, and NoSQL operator probes (`{ $ne: null }`) whose result sets are compared against a benign baseline query—suppressing false positives on endpoints that simply ignore the parameter.

H. CORS misconfiguration

The scanner issues cross-origin requests with an attacker-controlled `Origin` and flags reflection of the origin together with `Access-Control-Allow-Credentials: true`, as well as the invalid wildcard-plus-credentials combination and open origin reflection.

I. SSRF detection

For endpoints accepting URL-valued parameters, the scanner injects loopback and link-local addresses—including cloud-metadata endpoints (`169.254.169.254`)—and flags the endpoint when the response echoes the target’s own internal health body or metadata markers, proving the server fetched an internal resource on the attacker’s behalf (CWE-918) [7].

J. GraphQL and specification audit

The scanner probes `/graphql` with an introspection query and flags schema disclosure [8]. A final specification audit flags operations that declare no security requirement, missing `components.securitySchemes`, and live endpoints absent from the specification (OWASP API9: improper inventory management).

K. Security grade and OWASP mapping

Every scan is reduced to an SSL-Labs-style letter grade and score: each finding deducts weight (-20 CRITICAL, -10 HIGH, -5 MEDIUM, -2 LOW) from a base of 100, mapping $[90, 100] \rightarrow A$, $[80, 90) \rightarrow B$, $[70, 80) \rightarrow C$, $[60, 70) \rightarrow D$, else F . Each finding is tagged with its OWASP API Security Top 10 (2023) category and a plain-language remediation step, and the vulnerable endpoints are rendered as a hub-and-spoke attack-surface graph colored by severity.

VI. PRODUCT PLATFORM

The hosted backend hardens the scanner for real use: per-client rate limiting, optional API-key authentication (`SENTINEL_API_KEY`) and session login (`SENTINEL_USER/SENTINEL_PASSWORD`), a bounded worker pool (`MAX_WORKERS`) instead of unbounded threads, scan cancel and delete, HTML/JSON/PDF/SARIF/CSV report download, WebSocket live log streaming, a regression diff endpoint comparing successive scans of the same target, optional webhook delivery on completion, CORS, structured logging, and a richer health endpoint. The dashboard surfaces the grade badge, the attack-surface map, live search over findings, and one-click demo scanning behind a geometric robotic consent gateway (a counter-rotating hexagonal-aperture iris with a heads-up-display telemetry console); an in-app operator manual documents the fourteen detection classes and usage, and a downloadable preprint of this paper is served alongside the product.

VII. EVALUATION

We evaluated against a corpus of two deliberately vulnerable sandbox APIs—a “Bank” API and a “Library” API with different identifier names and flaw mixes—each including secure control endpoints (ownership-enforced reads, admin-gated stats) that must produce zero findings.

TABLE II
BENCHMARK RESULTS (GO ENGINE)

Metric	Bank	Library
Vulnerability classes	14	14
Seeded findings	32	10
Detected	32	10
False positives	0	0
False negatives	0	0

Aggregated over both targets, the scanner achieves **precision = recall = F1 = 1.0** (42/42 true positives, zero false positives, zero false negatives). The secure control endpoints

produced zero findings, confirming the absence of false positives on correctly implemented endpoints. An end-to-end test of the deployed product—scanning the live sandbox over the public Internet—returns all 32 findings with grade **F** (0/100), as expected for a deliberately vulnerable target.

VIII. NOVEL CONTRIBUTIONS

- **A spec-derived authorization oracle**—the security contract is encoded as a differential test, not guessed by fuzzing.
- **Generic principal-identifier inference**—no hardcoded field names, enabling generalization across API designs.
- **Recursive schema-contract diffing**—detects nested over-exposure, not just top-level field leaks.
- **Declared-vs-enforced security comparison**—detects broken authentication from the specification’s own security declarations.
- **Fourteen-class coverage with OWASP mapping and A–F grading**—including JWT, SQL/NoSQL injection, CORS, SSRF, and GraphQL checks, each contextualized and scored.
- **An active-probe oracle suite**—crafted JWT forgeries, SQL/NoSQL payloads, CORS origin reflection, SSRF loopback/metadata, and GraphQL introspection, with baseline-differential de-duplication to hold precision at 1.0.
- **Production exports and observability**—SARIF 2.1.0 [9] and CSV export, WebSocket live logs, and a regression diff between successive scans.
- **Explainability by construction**—every finding ships a reproduction and evidence diff, yielding zero false-positive noise.
- **A production-hardened, deployed platform**—auth, rate limiting, bounded concurrency, cancel/delete, and HTML/JSON/PDF reporting, live on Railway.
- **A two-target benchmark harness** reporting precision/recall/F1 against seeded ground truth.

IX. LIMITATIONS AND FUTURE WORK

The scanner currently models single-principal per-endpoint access; role lattices, session (shadow-token) handling, and side-channel timing remain future work, as do LLM-assisted oracles for business-logic leaks, CI/CD integration, and a Rust-class high-concurrency engine.

X. CONCLUSION

Athera Secure demonstrates that the most damaging API vulnerability classes can be detected deterministically, with high precision and full explainability, by treating API security as a differential-testing problem against the API’s own specification. The deployed system detects fourteen vulnerability classes with $F1 = 1.0$ across a two-target benchmark, grades every scan, maps findings to OWASP, exports to five report formats, and renders an interactive attack-surface map—packaged as a hardened, publicly reachable product.

REFERENCES

- [1] OWASP Foundation. *OWASP API Security Top 10:2023*. <https://owasp.org/API-Security/>
- [2] W. M. McKeeman, “Differential Testing for Software,” *Digital Technical Journal*, vol. 10, no. 1, pp. 100–107, 1998.
- [3] V. Atlidakis, P. Godefroid, and M. Polishchuk, “RESTler: Stateful REST API Fuzzing,” in *Proc. IEEE/ACM ICSE*, 2019.
- [4] A. Arcuri, “RESTful API Automated Test Case Generation with EvoMaster,” *ACM Trans. Softw. Eng. Methodol.*, vol. 28, no. 1, 2019.
- [5] Y. Liu *et al.*, “Morest: Model-based RESTful API Testing with Execution Feedback,” in *Proc. ISSSTA*, 2022.
- [6] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” IETF RFC 7519, 2015.
- [7] MITRE, “CWE-918: Server-Side Request Forgery (SSRF),” <https://cwe.mitre.org/data/definitions/918.html>
- [8] GraphQL Foundation, “GraphQL Specification: Introspection,” <https://spec.graphql.org>
- [9] OASIS, “Static Analysis Results Interchange Format (SARIF) Version 2.1.0,” <https://docs.oasis-open.org/sarif/sarif/v2.1.0/>